

C++ 为什么选择OOP

OOP是Object Oriented Program

1. 潘敏学老师邮箱:mxp@nju.edu.cn
 2. 不封装存在很大的安全隐患(数据暴露, 可以被直接修改)
 3. 不符合数据类型的定义,使用封装实现OOP
- [1. non-OO Solution 非面向对象的解决方案](#)
 - [2. OO Solution 面向对象的解决方案](#)
 - [3. OOP 面向对象](#)
 - [4. OOP评价标准](#)
 - [5. ENCAPSULATION\(封装\)](#)
 - [6. 对象类型的判断](#)
 - [6.1. 方法一:运行时判断](#)
 - [6.2. 方法二:编译时判断](#)

1. non-OO Solution 非面向对象的解决方案

```
1 //non-OO Solution
2 #include <stdio.h>
3 #define STACK_SIZE 00
4 struct Stack{
5     int top;
6     int buffer[STACK_SIZE];
7 };
8 //push和stack是相关的, 但是不是显式相关
9 bool push(Stack &s, int i){
10     if(s.top == STACK_SIZE - 1) {
11         printf("Stack is overflow.\n");
12         return false;
13     }else{
14         s.top++;
15         s.buffer[s.top] = i;
16         return true;
17     }
18 }
19 bool pop(Stack &s, int &i){
20     if (s.top == -1){
21         printf("Stack is empty.\n");
22         return false;
23     }else{
24         i = s.buffer[s.top];
25         s.top--;
26         return true;
27     }
28 }
29 void main(){
30     Stack st1, st2;
31     st1.top = -1; //安全隐患
```

```

32     st2.top = -1; //安全隐患
33     int x;
34     push(st1, 12);
35     pop(st1, x);
36     //可以直接操控其中的数据
37     st1.buffer[2] = -1; //违背ADT
38     st2.buffer[2]++;    //违背ADT
39 }

```

2. OO Solution 面向对象的解决方案

1. cfront:用来进行检查一些访问权限的问题。

```

1  #include <iostream.h>
2  #define STACK_SIZE 100
3  class Stack
4  {   private:
5       int top;
6       int buffer[STACK_SIZE];
7   public:
8       Stack(){ top = -1; } //定义的构造方法
9       bool push(int i);
10      bool pop(int& i);
11  };
12  bool Stack::push(int i){
13      if (top == STACK_SIZE-1) {
14          cout << "Stack is overflow.\n";
15          return false;
16      }else{
17          top++;
18          buffer[top] = i;
19          return true;
20      }
21  }
22  bool Stack::pop(int& i){
23      if (top == -1) {
24          cout << "Stack is empty.\n";
25          return false;
26      }else {
27          i = buffer[top];
28          top--;
29          return true;
30      }
31  }
32  void main()
33  {   Stack st1, st2;
34      int x;
35      st1.push(12);
36      st1.pop(x);
37      //st1.buffer[2] = -1无法操作
38      //cfront用来检查
39  }

```

1. 实际上,程序存储的时候并没有发生变化

```

1 struct Stack{
2     int top;
3     int buffer[STACK_SIZE];
4 };
5 //this是指向自己的指针
6 //对象的函数至少都持有一个this
7 bool push(Stack *const this,int i){
8     if (top == STACK_SIZE-1) {
9         cout << "stack is overflow.\n";
10        return false;
11    }else{
12        top++;
13        buffer[top] = i;
14        return true;
15    }
16 }
17 bool pop(Stack *const this,int& i){
18     if (top == -1) {
19         cout << "Stack is empty.\n";
20         return false;
21     }else {
22         i = buffer[top];
23         top--;
24         return true;
25     }
26 }
27 void main(){
28     Stack st1, st2;
29     st1.top = -1;
30     st2.top = -1;
31     int x;
32     push(st1,12);
33     pop(st1,x);
34 }

```

3. OOP 面向对象

1. Concepts 面向对象概念

1. Program = Object₁ + Object₂ + ... + Object_n
2. 对象:数据 + 操作
3. 信息:函数调用
4. 类

2. Classify 分类

1. Object-Oriented 面向对象
2. Object-Based(Ada:基于对象的语言)
 - Without Inheritance

4. OOP评价标准

1. 高扩展性

2. 质量

- 外部评价指标：正确性、效率、健壮性、可靠性、可用性、可重用性
- 内部评价指标：可读性、可维护性、可移植性

5. ENCAPSULATION(封装)

具体到markdown文件中

6. 对象类型的判断

6.1. 方法一:运行时判断

1. 使用if..else

```
1  int i;
2  if(typeid(i) == typeid(int) )
3      cout << "i is int" << endl ;
4  else
5      cout << "i is not int" << endl ;
```

6.2. 方法二:编译时判断

```
1  template<class T>
2  void func(T t ){
3      cout << "i is not int" << endl ;
4  }
5  template<> void func<int>(int i){//特化
6      cout << "i is int" << endl ;
7  }
8  int i;
9  func(i)
```